

Nonlinear pipeline

Nonlinear pipeline is a dynamic pipeline, reconfigure to evaluate multiple functions at different times and so has more than one reservation table as shown in Fig 2, one for one function evaluation.

Linear pipeline is a static pipeline, reconfigure to evaluate a single function and so has a single reservation table as shown in Fig 1.

A dynamic pipeline allows feed-forward and feedback connection in addition to the streamline connection (Fig 2).

A static pipeline allows only streamline connection (Fig 1)

A static pipeline partitions a given function into a sequence of linearly ordered sub-functions as it has only streamline connection. In dynamic pipeline it is difficult as the pipeline segments are interconnected with loops due to feedback connections in addition to streamline connections

All instructions to a static pipeline use the same reservation table whereas dynamic pipeline may allow different initiations to follow a mix of reservation tables

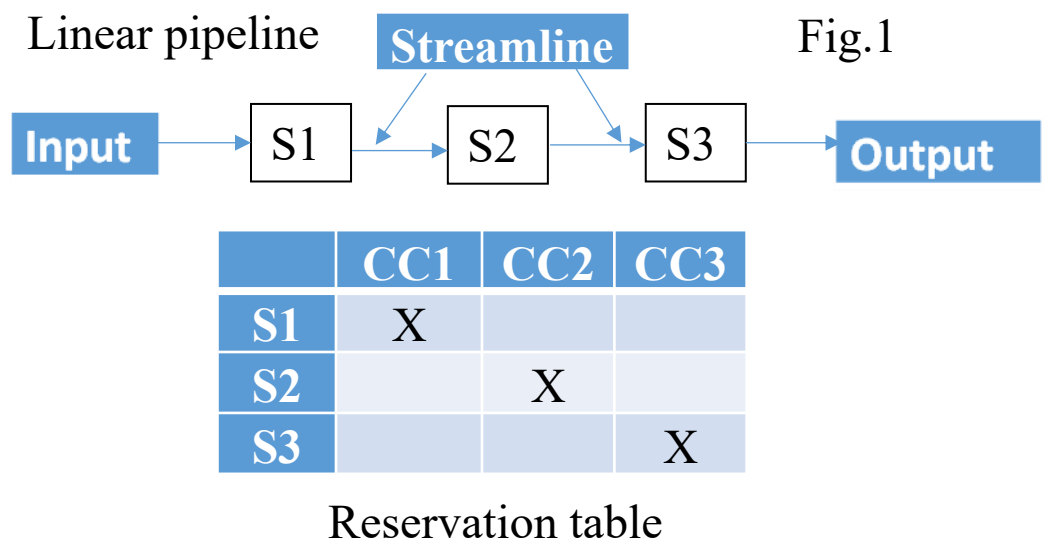
Example: Fig 1 shows a linear pipeline having 3 segments (S1, S2, S3).

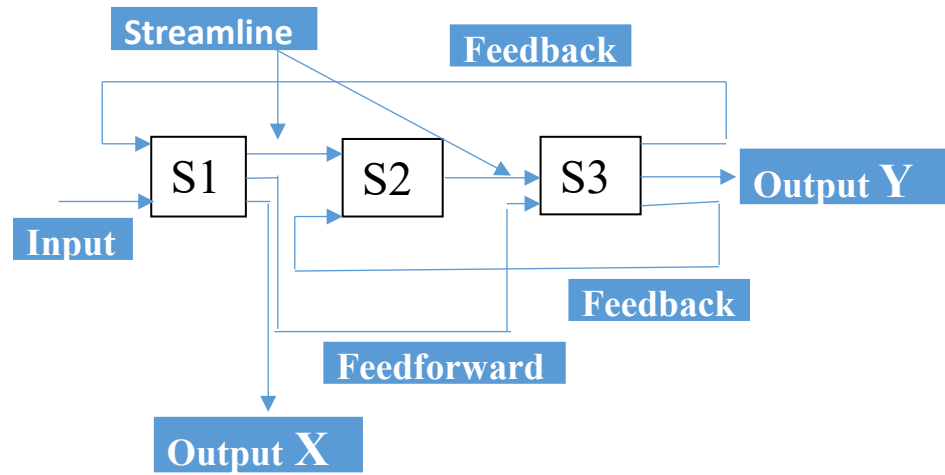
Segments are connected with each other using streamline connection.

Evaluate a single function.

Function enters into the segment S1 of the pipeline (shown as Input in Fig 1), output of the function is obtained from segment S3 of the pipeline.

Function takes 3 clock cycles for evaluation.





Nonlinear pipeline having 3 segments

Fig.2

Reservation table for function X: $S1 \rightarrow S2 \rightarrow S3 \rightarrow S2 \rightarrow S3 \rightarrow S1 \rightarrow S3 \rightarrow S1$ is the movement of X through the pipeline

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8
S1	X					X		X
S2		X		X				
S3			X		X		X	

Reservation table for function Y: $S1 \rightarrow S3 \rightarrow S2 \rightarrow S3 \rightarrow S1 \rightarrow S3$ is the movement of Y through the pipeline

	CC1	CC2	CC3	CC4	CC5	CC6
S1	Y				Y	
S2			Y			
S3		Y		Y		Y

Segments are connected with each other using streamline, feedback and feed forward connection.

Two feedback connections, from S3 to S1 and S3 to S2.

One feedforward connection, from S1 to S3.

Evaluates two functions, X and Y. Both the functions X and Y enter into the segment S1 of the pipeline. But output X is obtained from segment S1 whereas output Y is obtained from segment S3 of the pipeline.

Fig 2 also shows the reservation table for evaluating these functions. The function X takes 8 clock cycles (CC1 to CC8) and the function Y takes 6 clock cycles (CC1 to CC6) for evaluation.

Latency analysis: Latency is the delay that occurs between the start of 2 successive pipeline operations (i.e. 2 successive initiations in a pipeline). The number of clock cycles between 2 successive initiations is the latency of the pipeline. So latency of K means that 2 successive initiations in the pipeline are separated by K number of clock cycles. For example, Fig 3 shows the space time diagram for 2 successive initiations of function X (X1 and X2) in the nonlinear pipeline which is shown in Fig 2. It is drawn by considering the movement of X through the different segments of the pipeline as shown in the reservation table of function X (i.e. $S1 \rightarrow S2 \rightarrow S3 \rightarrow S2 \rightarrow S3 \rightarrow S1 \rightarrow S3 \rightarrow S1$).

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12	CC13	CC14	CC15	CC16
S1	X1					X1		X1	X2					X2		X2
S2		X1		X1						X2		X2				
S3			X1		X1		X1				X2		X2		X2	

Fig 3: Space time diagram for 2 initiations, X1 and X2

It can be observed from Fig 3 that the first initiation of function X i.e. X1 enters into the segment S1 of the pipeline in CC1 whereas the second initiation of function X i.e. X2 enters into the segment S1 of the pipeline in CC9. So the latency is $9-1=8$ clock cycles in this case. Moreover, the evaluation of function X needs 8 clock cycles as observed from its reservation table in Fig 2. So the evaluation of X1 is over in CC8 and the evaluation of X2 is over in CC16. So $8 \times N$ number of clock cycles are required to complete the evaluation of N number of different initiations of function X (X1, X2, X3,.....XN).

Forbidden latency:

- Collision occurs in the pipeline if 2 or more initiations use the same pipeline segment at the same time that is in the same clock cycle. The latency that causes collision is called forbidden latency.
- The distance between any 2 check marks (for example, X in the reservation table of function X, Y in the reservation table of function Y in Fig 2) in the same row of the reservation table is forbidden latency.
- For example, let us consider the reservation table of function X. In the first row of this table there are 3 check marks in CC1, CC6, CC8. So the forbidden latencies are $6-1=5$, $8-6=2$, $8-1=7$. In the second row of this table there are 2 check marks in CC2 and CC4. So the forbidden latency is $4-2=2$. In the third row of this table there are 3 check marks in CC3, CC5, CC7. So the forbidden latencies are $5-3=2$, $7-5=2$, $7-3=4$. So the function X has 4 forbidden latencies and these are 2, 4, 5, 7.
- Similarly the forbidden latencies for function Y can be evaluated from its reservation table. The function Y has 2 forbidden latencies. These are 2, 4.

← latency=2 →

	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11
S1	X1		X2		X3	X1	X4	X1X2		X2X3	...
S2		X1		X1X2		X2X3		X3X4		X4	...
S3			X1		X1X2		X1X2X3		X2X3X4		...

Fig 4: Space time diagram for function X with latency equal to 2

Three initiations of function X (X1, X2, X3) with latency 2 are shown in the space time diagram of Fig 4. X1 enters into the pipeline in CC1, X2 enters into the pipeline just after 2 clock cycles from the initiation of X1, X3 enters into the pipeline just after 2 clock cycles from the initiation of X2. Collision occurs in CC4, CC5, CC6, CC7, CC8, CC9, CC10 as 2 is a forbidden latency.

Two initiations of function X (X1, X2) with latency 5 are shown in the space time diagram of Fig 5. X1 enters into the pipeline in CC1, X2 enters into the pipeline just after 5 clock cycles from the initiation of X1 i.e. in CC6. Both X1 and X2 occupy the segment S1 of the pipeline in CC6 and hence cause collision.

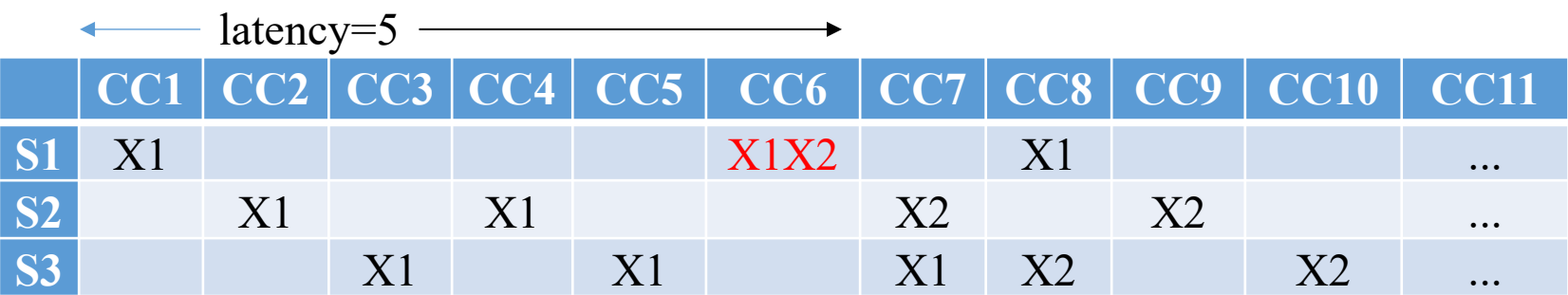


Fig 5: Space time diagram for function X with latency equal to 5

Observation from Fig 3: It can be observed from the space time diagram in Fig 3 that the latency 8 is a safe latency as it causes no collision. The same is true for any latency which is greater than 8. So latency value greater than equal to 8 does not cause any collision for function X. But if the value of N that is the number of initiations is 100 and latency is 8 then 100×8 clock cycles are required to complete the evaluation of 100 tasks. So it reduces throughput (output as obtained from the pipeline in unit time i.e. per clock cycle) of the pipeline. To improve the pipeline performance that is to increase the pipeline throughput latency should be as small as possible at the same time such latency does not cause any collision.

Latency sequence: A sequence of permissible non-forbidden latencies between successive initiations. The permissible non-forbidden latency is the delay that can be used between the start of 2 successive pipeline operations (i.e. 2 successive initiations in a pipeline) without causing any collision.

Latency cycle: Latency cycle is the latency sequence which repeats the same subsequence (cycle) indefinitely.

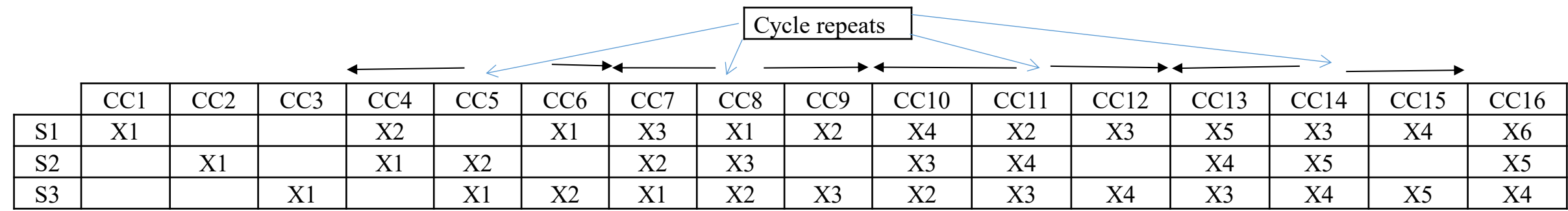
	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12	CC13	CC14	CC15	CC16	CC17	CC18	CC19
S1	X1	X2				X1	X2	X1	X2	X3	X4				X3	X4	X3	X4	X5
S2		X1	X2	X1	X2						X3	X4	X3	X4					
S3			X1	X2	X1	X2	X1	X2				X3	X4	X3	X4	X3	X4		

Latency cycles in using the pipeline in Fig 2 to evaluate the function X without causing a collision

- Latency cycle (1,8): The latency cycle (1, 8) represents the infinite latency sequence 1, 8, 1, 8,The successive initiations of new tasks are separated by 1 clock cycle and 8 clock cycle alternately.

For example five initiations of function X (X1, X2, X3, X4, X5) are shown in the above space time diagram. The diagram is drawn for 19 clock cycles (CC19). It can be extended for more clock cycles. The latency between X1 (enters into the pipeline in CC1) and X2 (enters into the pipeline in CC2) is 1, the latency between X2 (enters into the pipeline in CC2) and X3 (enters into the pipeline in CC10) is 8, again the latency between X3 (enters into the pipeline in CC10) and X4 (enters into the pipeline in CC11) is 1, again the latency between X4 (enters into the pipeline in CC11) and X5 (enters into the pipeline in CC19) is 8.

2. Latency cycle (3): The successive initiations of new tasks are separated by 3 clock cycles.



For example, six initiations of function X (X1, X2, X3, X4, X5, X6) are shown in the above space time diagram. The diagram is drawn for 16 clock cycles (CC16). It can be extended for more clock cycles. The latency between X1 (enters into the pipeline in CC1) and X2 (enters into the pipeline in CC4) is 3, the latency between X2 (enters into the pipeline in CC4) and X3 (enters into the pipeline in CC7) is 3, again the latency between X3 (enters into the pipeline in CC7) and X4 (enters into the pipeline in CC10) is 3, again the latency between X4 (enters into the pipeline in CC10) and X5 (enters into the pipeline in CC13) is 3 and so on.

3. Latency cycle (6): The successive initiations of new tasks are separated by 6 clock cycles.

	Cycle repeats ← → 															
	CC1	CC2	CC3	CC4	CC5	CC6	CC7	CC8	CC9	CC10	CC11	CC12	CC13	CC14	CC15	CC16
S1	X1					X1	X2	X1				X2	X3	X2		
S2		X1		X1				X2		X2				X3		X3
S3			X1		X1		X1		X2		X2		X2		X3	

For example, three initiations of function X (X1, X2, X3) are shown in the above space time diagram. The diagram is drawn for 16 clock cycles (CC16). It can be extended for more clock cycles. The latency between X1 (enters into the pipeline in CC1) and X2 (enters into the pipeline in CC7) is 6, the latency between X2 (enters into the pipeline in CC7) and X3 (enters into the pipeline in CC13) is 6, and so on.

Average latency: The average latency of a latency cycle is obtained by dividing the sum of all latencies by the number of latencies along the cycle. The latency cycle (1, 8) has an average latency = 4.5.

Constant cycle: The constant cycle is a latency cycle having only 1 latency value. The latency cycles 3 and 6 are the example of constant cycle. The cycle 3 has latency value 3 (constant latency) and the cycle 6 (constant latency) has latency value 6.

Collision free scheduling

Scheduling is the initiation of different tasks in the pipeline i.e. the different tasks are scheduled in the pipeline for execution. Objective of scheduling events in a pipeline is to obtain the shortest average latency (to reduce execution time and increase pipeline throughput) between successive initiations without any collision.

Method for achieving such collision free scheduling for function X

This method uses collision vector, state diagram, simple cycles and greedy cycles to compute the minimal average latency (MAL).

Collision vector:

Maximum forbidden latency is m and $m \leq n-1$ where n is the number of columns in the reservation table

Permissible latency is P , P ($1 \leq P \leq m-1$) should be as small as possible to reduce the overall execution time and to increase the pipeline throughput. $P=1$ is the ideal case for static pipeline.

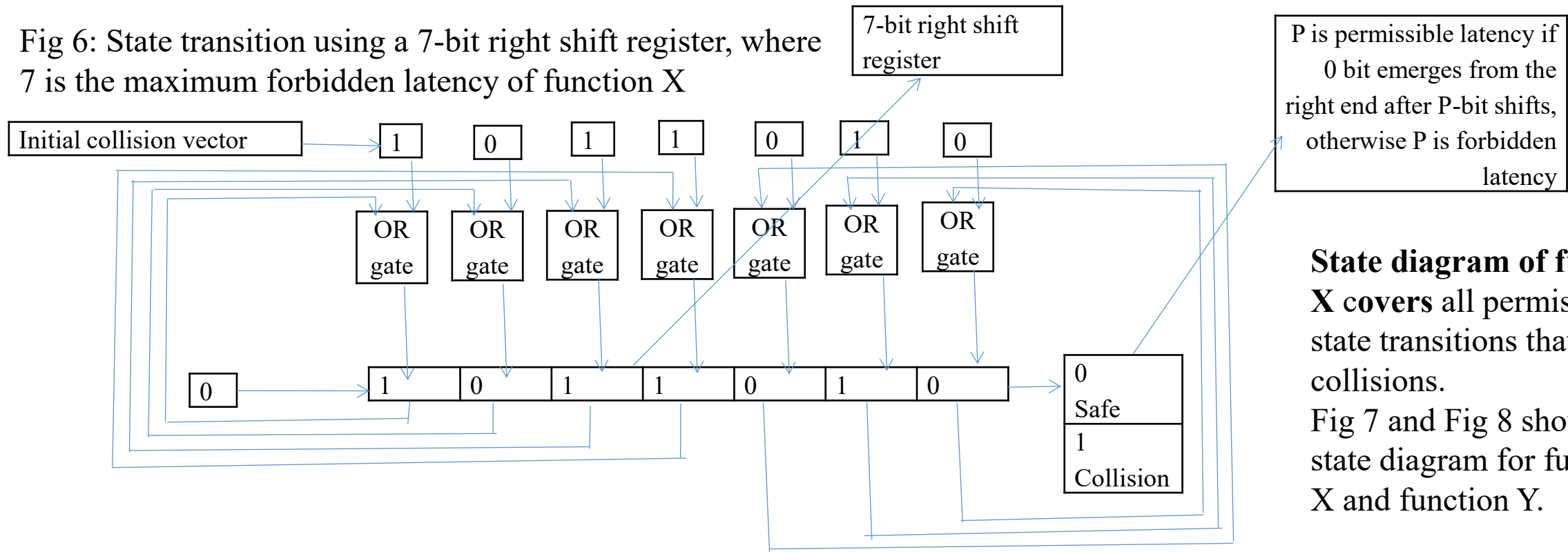
Collision vector (C) is a m -bit binary vector, $C=(C_m C_{m-1} \dots C_2 C_1)$, the value of C_i (i^{th} bit of collision vector, $1 \leq i \leq m$) is 1 if latency i is forbidden and 0 if latency i is permissible

Collision vector for function X (C_X) [function Y (C_Y)]

For function X, $n=8$, $m \leq 8-1$ i.e. the maximum value of m is 7, $1 \leq P \leq 7-1$ i.e. the maximum value of P is 6.

Collision vector for function X (C_X) is a 7-bit vector, C_X is 1011010. [For function Y, $n=6$, $m \leq 6-1$ i.e. the maximum value of m is 5, $1 \leq P \leq 5-1$ i.e. the maximum value of P is 4. Collision vector for function Y (C_Y) is a 5-bit vector, is 1010.]

Fig 6: State transition using a 7-bit right shift register, where 7 is the maximum forbidden latency of function X



State diagram of function X covers all permissible state transitions that avoids collisions.

Fig 7 and Fig 8 show the state diagram for function X and function Y.

Observation from Fig 6

Initial content of the shift register is C_x i.e. 1011010. Logical 0 enters from left end of the register during right shift operation. Logical 1 emerges from the right end of the register if the content of the register is shifted by 2, 4, 5, 7 (forbidden latency) number of times and logical 0 emerges from the right end of the register if the content of the register is shifted by 1, 3, 6 (permissible latency) number of times towards right.

7 two input OR gates are used. The initial collision vector is one of the inputs of the OR gates (fixed input) and the content of the shift register is the other input of the OR gates (changes after each shift operation). The output of the OR gates is the bitwise OR operation between the initial collision vector (1011010) and the content of the shift register (1011010). So the output of the OR gates is 1011010. This output decides the new content of the shift register.

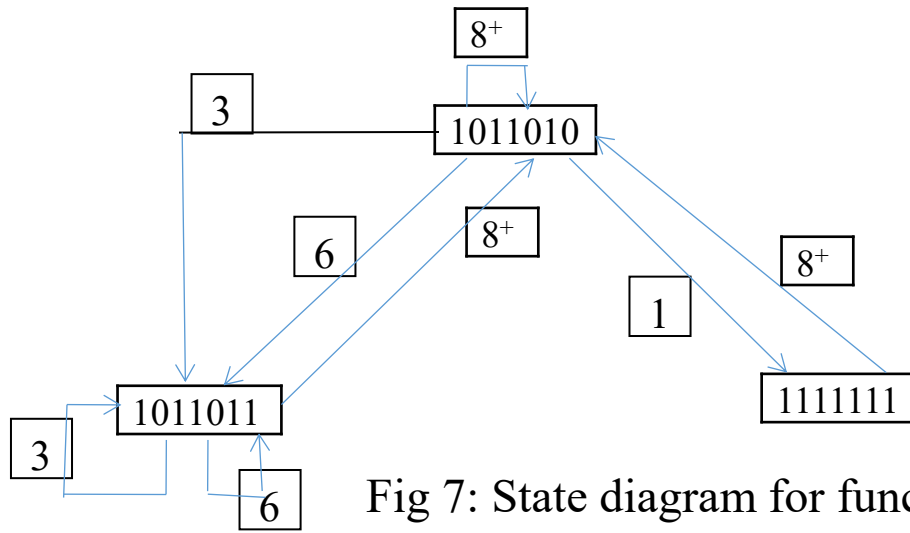


Fig 7: State diagram for function X

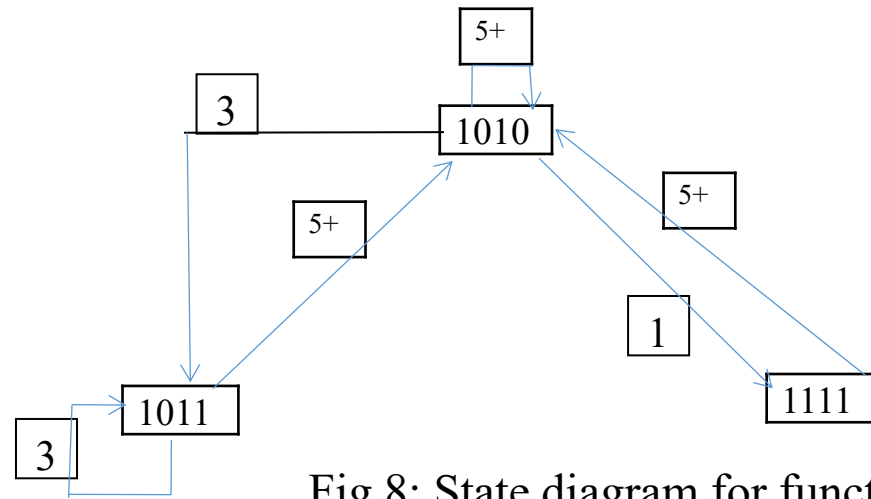


Fig 8: State diagram for function Y

Observation from Fig 7

Three states, 1011010, 1011011, 1111111 and the transitions among the states are shown in Fig 7.

Details of state transitions

1. Present state **1011010** to next state **1011011**

This state transition occurs for $P=3$ and $P=6$. The present content of the shift register is 1011010 and the present state is 1011010 for both the values of P . Next state is 1011011.

2. Present state **1011011** to next state **1011010**

This state transition occurs for $P=8^+$ (indicate $P \geq 8$ as 8, 9, 10 etc. are permissible latencies for function X but not used at the time of scheduling tasks in the pipeline due to huge delay that is low throughput). The present content of the shift register is 1011011 and the present state is 1011011. Next state is 1011010.

3. Present state 1011011 to next state 1011011

This state transition occurs for $P=3$ and $P=6$. The present content of the shift register is 1011011 and the present state is 1011011 for both the values of P . Next state is 1011011 for both the values of P .

4. Present state 1011010 to next state 1011010

This state transition occurs for $P=8^+$. The present content of the shift register is 1011010 and the present state is 1011010. Next state is 1011010.

5. Present state 1011010 to next state 1111111

This state transition occurs for $P=1$. The present content of the shift register is 1011010 and the present state is 1011010. Next state is 1111111.

6. Present state 1111111 to next state 1011010

This state transition occurs for $P=8^+$. The present content of the shift register is 1111111 and the present state is 1111111. Next state is 1011010.

Observation:

From the initial state (1011010), only 3 outgoing transitions are possible, corresponding to the 3 permissible latencies 6, 3, 1.

Similarly from the state 1011011, one reaches the same state after either 3 shifts or 6 shifts.

Finally when the number of shifts is $m+1$ or more, all transitions are redirected back to the initial state. For example, after 8 or more shifts (denoted by 8^+) the next state must be the initial state regardless of which state the transition starts from.

Cycle, simple cycle and greedy cycle

Cycle: An infinitely many latency cycles one can trace from the state diagram (Fig 7). Few example of such cycles are (1, 8), (1, 8, 6, 8), (3), (6), (3, 8), (3, 6, 3),....

Simple cycle: It is a latency cycle in which each state appears only once. So (1, 8), (3), (6), (3, 8), (8) and (6, 8) are simple cycles.

The cycle (1, 8, 6, 8) traverses the state 1011010 multiple times. It starts from the state 1011010, then goes to the state 1111111 for $P=1$, then comes back to the state 1011010 for $P=8^+$, then goes to the state 1011011 for $P=6$, then comes back to the state 1011010 for $P=8^+$. So this cycle is not a simple cycle.

The cycle (3, 6, 3, 8, 6) traverses the state 1011011 multiple times. It starts from the state 1011011, then goes to the state 1011011 for $P=3$, then goes to the state 1011011 for $P=6$, then goes to the state 1011011 for $P=3$, then goes to the state 1011010 for $P=8$, then comes back to the state 1011011 for $P=6$.

Greedy cycle: Greedy cycle is one whose edges are all made with minimum latencies from their respective starting states. Greedy cycles are those simple cycles whose average latencies must be lower than those of other simple cycles. The cycles (1, 8) and (3) are greedy cycles for function X as these cycles have minimum latencies than the other simple cycles. [(1, 5) and (3) are greedy cycles for function Y]

The simple cycle (1, 8) is a greedy cycle as its average latency ($\bar{L} = 4.5$) is lower than that of the simple cycle (6, 8) (average latency $\bar{L} = 7$) and (3, 8) (average latency $\bar{L} = 5.5$).

The simple cycle (3) is a greedy cycle as its constant latency (3) is lower than that of the other constant cycles.

Finally the constant latency 3 (as 3 is less than 4.5 that is the latency of the other greedy cycle) is equal to MAL for evaluating function X without any collision.